Introduction to Computing (CS 101)

Relational Expressions

T. Venkatesh & John Jose

Department of Computer Science & Engineering

Indian Institute of Technology Guwahati

Relational Operators

Operator	Meaning
<	Less than
>	Greater than
<=	less than or equal to
>=	greater than or equal to
==	is equal to
!=	is not equal to

- Relational expressions evaluate to the integer values 1 (true) or 0 (false).
- All of these operators are called binary operators because they take two expressions as operands.

Practice with Relational Expressions

- `int a = 1, b = 2, c = 3 ;`

Expression	Value
`a < c`	
`b <= c`	
`c <= a`	
`a > b`	
`b >= c`	
`(a + b)>= c`	
`(a + b)== c`	
`a != b`	
`(a + b)!= c`	

Practice with Relational Expressions

- `int a = 1, b = 2, c = 3 ;`

Expression	Value
`a < c`	T (T=1, F=0)
`b <= c`	T
`c <= a`	F
`a > b`	F
`b >= c`	F
`(a + b)>= c`	T
`(a + b)== c`	T
`a != b`	T
`(a + b)!= c`	F

Arithmetic Expressions: True or False

- Arithmetic expressions evaluate to numeric values.
- An arithmetic expression that has a value of zero is false.
- An arithmetic expression that has a value other than zero is true.

Practice with Arithmetic Expressions

- `int a = 1, b = 2,c = 3;`
- `float x = 3.33, y = 6.66;`

Expression	Numeric Value	True/False
`a + b`	3	T
`b-2*a`	0	F
`c- b-a`	0	F
`c-a`	2	T
`y-x`	3.33	T
`y-2*x`	0.0	F

Structured Programming

- All programs can be written in terms of only three control structures
  - Sequence, selection and repetition
- **The sequence structure**
  - Unless otherwise directed, the statements are executed in the order in which they are written.
- **The selection structure**
  - Used to choose among alternative courses of action.
- **The repetition structure**
  - Allows an action to be repeated while some condition remains true.

Selection: the if statement

```
  if  ( condition )
{
 statement(s) /* body of the if
                 statement */

}
```

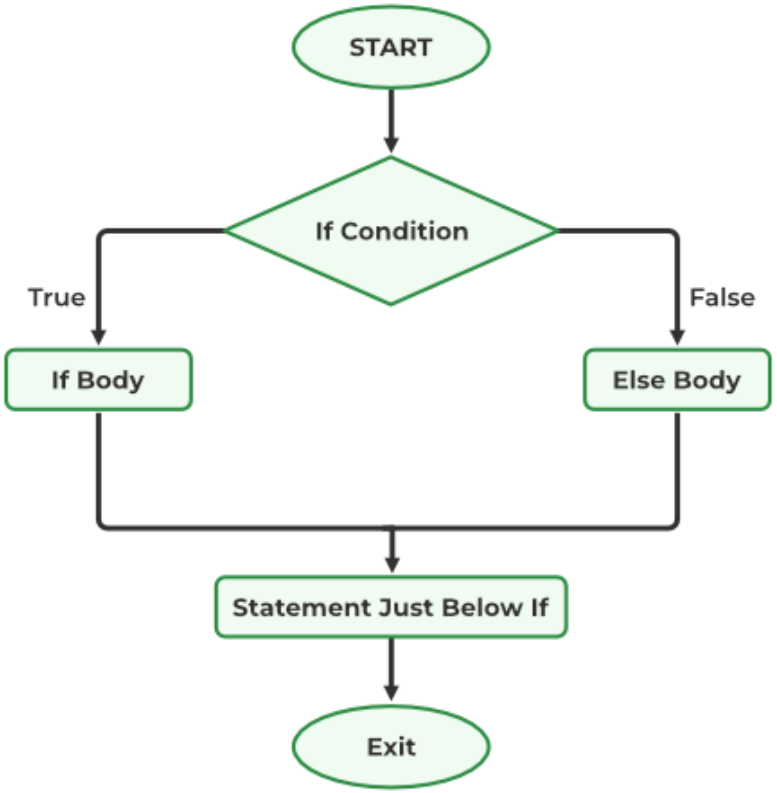
- The braces are not required if the body contains only a single statement.
- Recommended to keep { }by the C Coding Standards.
- Indent the body of the if statement 3 to 4 spaces -- be consistent!

Examples

```
if ( age >= 18 )
{
    printf(“Vote!\n”) ;
}
if ( value == 0 )
{
  printf(“You entered a zero.\n”) ;
  printf (“Please try again.\n”) ;
}
```

Selection: the if-else statement

```
if  (condition )
{
  statement(s)
}
else
{
      statement(s)
}
```



Example

```
if ( age >= 18 )
{
    printf(“Vote!\n”) ;
}
else
{
    printf(“Maybe next time!\n”) ;
}
```

Nesting of if-else Statements

```
if ( condition1 )
{
     statement(s)
}
else if ( condition2 )
{
     statement(s)
}
. . . /* more else clauses may be here */
else
{
     statement(s)  /* the default case */
}
```

Good Example : 2 if 2 else

```
if ( value == 0 )
{
    printf("Value you entered was 0\n");
}
else if ( value < 0 )
{
    printf("%d is negative.\n", value) ;
}
else
{
    printf("%d is positive.\n", value) ;
}
```

Bad Example : 2 if 1 else

```
 if ( n > 0 )
    if ( a > b  )
        z=a;
 else
     z=b;
```

```
if ( n > 0 )
  if ( a > b )
      z=a;
  else
      z=b;
```

```
if ( n > 0 )
{
  if (a> b)
      z=a;
 }
 else
     z=b;
```

Indentation will not ensure result :

else match with closest if

Code of Red box behaves like Code of Green box

= versus ==

```
int  a = 2 ;
if ( a = 1 )/* semantic(logic) Err! */
{
    printf (“a is one\n”) ;
}
else if (a == 2 )
{
    printf (“a is two\n”) ;
}  else {
    printf (“a is %d\n”, a) ;
}
```

= versus ==

- The statement if (a = 1) is syntactically correct, so no error message will be produced. (Some compilers will produce a warning.) However, a semantic (logic) error will occur.
- An assignment expression has a value, the value being assigned. In this case the value being assigned is 1, which is true.
- If the value being assigned was 0, then the expression would evaluate to 0, which is false.
- This is a VERY common error. So, if your if-else structure always executes the same, look for this typographical error.

Logical Operators

- So far we have seen only **simple conditions**.

  if (count > 10) . . .

- Sometimes we need to test multiple conditions in order to make a decision.
- **Logical operators** are used for combining simple conditions to make **complex conditions**.

&&	AND	if (x > 5 && y < 6)
||	OR	if (z == 0 || x > 10)
!	NOT	if (! (bob > 42))

Example Use of &&

```
if ( age < 1 && gender == 'm')
{
    printf ("Infant boy\n") ;
}
```

Truth Table for &&

$Expression_1$	$Expression_2$	$Expression_1$ && $Expression_2$
0	0	0
0	nonzero	0
nonzero	0	0
nonzero	nonzero	1

Exp_1 && Exp_2 && … && Exp_n will evaluate to 1 (true) only if ALL **sub-conditions** are true.

Example Use of ||

```
if (grade=='E' || grade=='F')
{
 printf("See you next semester!\n") ;
}
```

Truth Table for ||

$Expression_1$	$Expression_2$	$Expression_1$ || $Expression_2$
0	0	0
0	nonzero	1
nonzero	0	1
nonzero	nonzero	1

Exp_1 || Exp_2 || … || Exp_n will evaluate to 1 (true) if at least ONE sub-condition is true.

Example Use of !

```
if (!(x==2)) /* Same as (x!=2) */
{
  printf("x is not equal to 2") ;
}
```

Truth Table for !

Expression	! Expression
0	1
nonzero	0

Operator Precedence and Associativity

Precedence	Associativity
( )	left to right/inside-out
++ -- **!** + (unary) - (unary)	right to left
* / %	left to right
+ (addition) - (subtraction)	left to right
< <= > >=	left to right
== !=	left to right
&&	left to right
‖	left to right
= += -= *= /= %=	right to left

Example Programs

```
// Check whether an integer is odd or even
#include <stdio.h>
int main() {
    int number;
    printf("Enter an integer: ");
    scanf("%d", &number);
    // True if the remainder is 0
    if  (number%2 == 0) {
        printf("%d is an even integer.",number);
    }
    else {
        printf("%d is an odd integer.",number);
    }
    return 0;
}
```

Example Programs

```
// Program to relate two integers using =, > or <
#include <stdio.h>
int main() {
    int number1, number2;
    printf("Enter two integers: ");
    scanf("%d %d", &number1, &number2);
    if(number1 == number2) {
        printf("Result: %d = %d",number1, number2);
    }
    else if (number1 > number2) {
        printf("Result: %d > %d", number1, number2);
    }
    else {
        printf("Result: %d < %d",number1, number2);
    }
    return 0;
}
```

Example Programs

```
// To check whether the person is eligible to vote
#include <stdio.h>
int main()
{
    int p1_age, p2_age;
    printf("Enter the age of two persons: ");
    scanf("%d %d", &p1_age, &p2_age);
    if (p1_age < 18)
        printf("Person 1 is not eligible to vote.\n");
    else
        printf("Person 1 is eligible to vote.\n");
    if (p2_age < 18)
        printf("Person 2 is not eligible to vote.\n");
    else
        printf("Person 2 is eligible to vote.");
     return 0;
}
```

Thanks